

Localizing the cause of agent failures by intervention

An applied research report on **culprit**, a root-cause localization tool for Arize Phoenix.

Laxya Kumar September 2026 github.com/Lkumar209/culprit-agent-rca

Main finding. On 547 failed agent runs with known ground truth, counterfactual intervention – repairing a span and replaying the run – identifies the causal span in **1.000** of cases at **3.74 replays**, while every inspection-based method tops out near **0.54**, including a whole-trace LLM judge that is six times better than chance and shows no weakness on silent faults. The advantage holds across both fault classes, across two independent agent implementations (rule-based and a live LLM agent), and comes with an ability no inspection method has: declining to name anyone on runs that did not fail (97.8-100% vs 0%).

Why inspection plateaus there is a tested mechanism rather than an open question. Inspection is bounded by whether the trace contains a **contradiction** – not by fault visibility (no silent/overt gap at all) and not by which side of the agent/tool boundary the fault sits on. Two observation-phase faults that are identical in phase and both silent, differing only in whether the response contradicts its own request, score **1.000 and 0.000**. Intervention is 1.000 on both, because it generates the missing evidence rather than searching for it. The prediction was stated before the fault that tests it was written.

1. The question

2026 is the year agents went to production. Every observability vendor ships agent tracing now, and every framework emits spans. The effect is that an engineer whose multi-step agent returns a wrong answer has more telemetry than ever and still no answer to the only question they have: which step broke it?

Phoenix records what an agent did. When a multi-step run fails, it shows the whole trace tree and tells you the run failed. It does not tell you *which step caused it*, and in a trace of a dozen spans that is the only question the engineer actually has.

The obvious approach is to look: find the span with the error on it, or ask a model which span looks wrong. Both are **descriptive** — they score spans by appearance. The alternative is **causal**: repair a span, replay the run forward from it, and see whether the outcome moves. A span that changes the outcome is implicated; a span that does not is a bystander, however suspicious it looks.

The research question is whether that distinction is worth what it costs. Inspection is one API call or none. Intervention is several replays of the agent. If looking works, intervening is an expensive way to buy the same answer.

2. Why the question is not trivially answered

In a multi-step agent, the symptom and the cause are usually different spans. A tool that returns a truncated page, a stale number, or a plausible-but-wrong entity id produces a span that is *locally unremarkable*: well-formed data, sensible values, `status: OK`. The damage surfaces several steps later, in the final answer.

This gives the central distinction of the whole project:

- **Overt** faults announce themselves. The span carries an error. Symptom and cause are co-located.
- **Silent** faults return well-formed, wrong data. Nothing in the span looks unusual in isolation.

If silent faults are common, methods that score spans by appearance are searching the wrong place, and the cost of intervention might be justified.

3. Building something that can answer it

The hard part of this project was not the method. It was constructing a benchmark whose numbers mean anything.

The world. A synthetic business domain — vendors, orders, paginated invoices — deterministic from a seed, with amounts as integer cents so that rounding disagreements cannot be mistaken for injected faults. Vendor names share stems (`Northwind Inc.` / `Northwind LLC`) so that a wrong-entity fault has a *plausible* wrong answer rather than an obviously broken one.

The tasks. Four multi-hop question templates whose answers are computed programmatically: total quarterly spend for the vendor behind an order, policy cap checks, invoice counts, top quarter. Because the gold answer is computed rather than judged, "did the agent fail?" is a fact, not an opinion. Had task success itself been an LLM judgement, every number downstream would have inherited that judge's noise.

The agent. A tool-using policy that **derives every action from what it has observed**, never from the task's reference plan. This sounds like a detail and is the single load-bearing property of the benchmark: a policy that replayed the gold plan would sail through an injected fault unchanged, always produce the right answer, and every localization number would be measuring nothing. Because it reads observations, a corrupted `vendor_id` at step 1 genuinely poisons every later call. There is a test for exactly this.

The faults. Nine types, split silent/overt, injected one per run so the culprit span is known by construction. Six corrupt what a tool returns; three corrupt what the agent asks for.

The result: 1,046 traces — 547 failed with a known culprit (424 silent, 123 overt), 459 where a fault fired but the agent got the right answer anyway, and 40 clean runs. Median 11 spans.

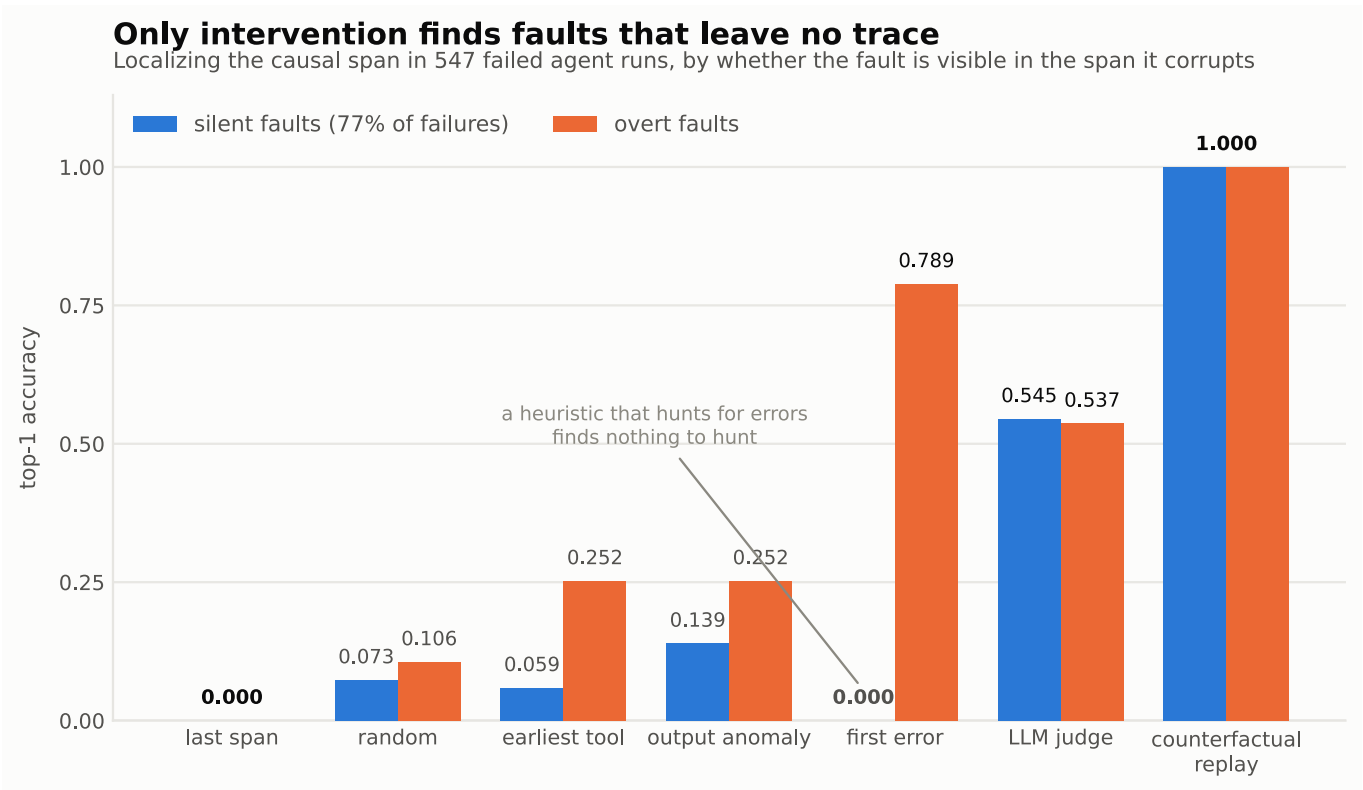
The method

For a span `k` : repair its output, replay forward, compare the final answer. Two search strategies sit on that primitive:

- **Exhaustive** — probe spans in trace order, stop at the first that moves the outcome. Earliest-first is not an arbitrary tie-break. When a fault propagates, repairing *any* downstream span carrying the corruption also changes the answer, so several spans are causally implicated and only the earliest is the root cause.
- **Bisection** — binary search on a monotone prefix probe. Writing `Q(k) = "shielding steps 0..k fixes the run"` , if the fault is at step `j` then `Q` is false below `j` and true at or above it. `Q` is a step function, the culprit is where it flips, and bisection finds it in $O(\log n)$ replays. Monotonicity is asserted for every fault type in the test suite.

4. What the experiments found

4.1 Inspection cannot see silent faults



method	top-1	silent (n=424)	overt (n=123)
last_span	0.000	0.000	0.000
earliest_tool	0.102	0.059	0.252
output_anomaly	0.165	0.139	0.252
first_error	0.177	0.000	0.789
cf_bisect	1.000	1.000	1.000

`first_error` — the heuristic every engineer actually uses — is respectable on overt faults and scores **exactly zero** on silent ones.

That result invites an obvious objection: of course an error-hunting heuristic finds nothing when there is no error. So I added `output_anomaly`, a domain-agnostic statistical detector that compares each tool call against the other calls of the same tool in the same trace, flagging numeric outliers and empty collections where siblings returned rows. It knows nothing about the domain or the fault taxonomy. It reaches 0.139 on silent faults.

What this establishes is narrower than "the information is missing", and the next section is why: a silent fault is invisible **in its own span**, not in the trace.

4.2 The LLM judge refuted my hypothesis

I predicted the judge would track `first_error`: fine on overt faults, near- useless on silent ones, because a silent fault leaves nothing locally visible.

method	top-1	silent	overt	cost
first_error	0.177	0.000	0.789	free
llm_trace_judge	0.543	0.545	0.537	1 call

No gap at all. The judge is not keying on error markers; it reconstructs the run's arithmetic from the trace and notices that the numbers do not add up. It is a genuinely good cheap localizer — roughly 3× the best heuristic for one API call — not the strawman I expected.

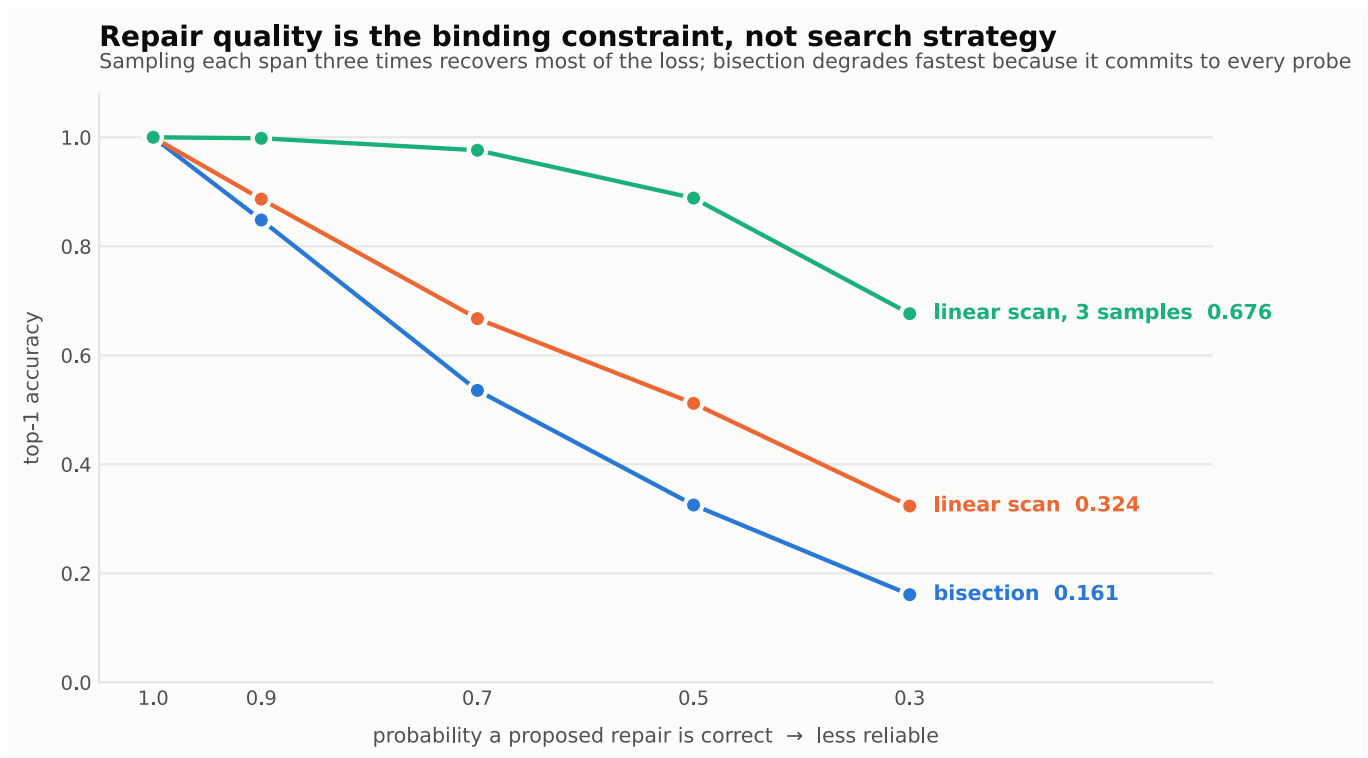
Two supporting results. The **per-span judge is worse than the whole-trace judge** (0.413 vs 0.476 on identical traces), which supports the locality argument even as it undercuts the blindness one: judging a span in isolation removes exactly the context needed to notice a plausible value is wrong. And the **direction of error** separates the two failure modes cleanly:

method	blames downstream	blames upstream	typical miss distance
first_error	100%	0%	7.72 spans
llm_trace_judge	57.6%	42.4%	3.26 spans

The distinction is in the *shape* of the error, not its size. `first_error` fails **systematically**: it always blames downstream, because with no error span to find it falls back to the end of the trace. The judge fails **unsystematically** — scattered in both directions, typically three spans off in a trace averaging 11.7 candidates. Neither is close. A judge that is wrong is not nearly right; it is wrong in a way you cannot correct by looking harder, which is what a systematic bias would let you do.

(Reported as mean *absolute* distance. The signed mean is +1.06, which looks like "off by one" and is an artifact of opposite errors cancelling — worth stating because it is exactly the statistic that would have flattered the judge.)

4.3 Repair reliability, not search, is the binding constraint

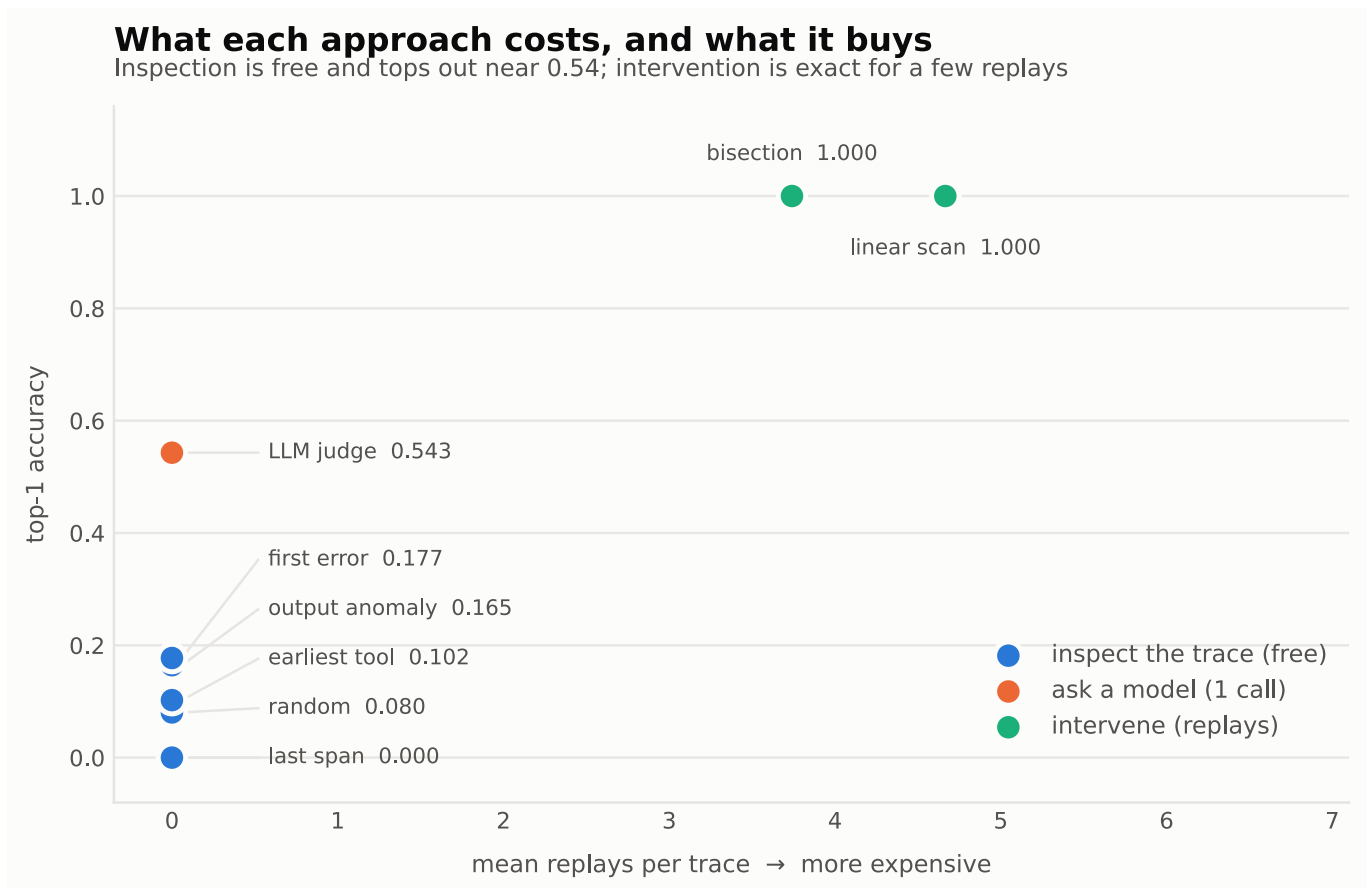


The 1.000 figures assume a perfect repair oracle. In production the repaired value comes from a model asked "what should this step have returned?", and that proposal is sometimes wrong. Sweeping that probability:

repair success	cf_bisect	cf_exhaustive	cf_exhaustive ×3
1.0	1.000	1.000	1.000
0.9	0.848	0.887	0.998
0.7	0.536	0.667	0.976
0.5	0.325	0.512	0.888
0.3	0.161	0.324	0.676

Accuracy is dominated by repair quality. Bisection degrades fastest, and the reason is structural: its efficiency comes from committing to each probe, which is exactly what hurts when probes are noisy. Sampling each span three times restores $0.667 \rightarrow 0.976$ at $p=0.7$ for $\sim 2.1\times$ the replays. **Tune the repair, not the search.**

4.4 Bisection is what to ship



cf_exhaustive pays for the culprit's depth; cf_bisect does not. Because faults here are injected at steps 0-3, culprits sit near the front and the linear scan is flattered. Repeating on deeper injection points:

method	culprits at steps 0-3	culprits at steps 2-7
cf_exhaustive	4.67 replays	7.70
cf_bisect	3.74	4.17

Bisection is cheapest, depth-independent, and equally accurate.

4.5 Two negative results worth reporting

Guided replay does not earn its cost. The idea was to let a cheap scorer order the probes. It was *wrong* as first implemented, and the bug is instructive: ordering by suspicion breaks the invariant that makes the scan correct. A confirmed hit proves only that the culprit is at or before that span, never that it *is* that span, because propagation implicates downstream spans too. Treating a confirmation as a verdict cost 23 points of top-1. Corrected — confirmation as an upper bound, then bisect below it, with a full-bisection fallback when the shortlist misses — accuracy returns to 1.000 but costs 5.7-7.1 replays against plain bisection's 3.74. This held with the LLM judge as scorer. A scorer only pays if it is cheaper than the search it replaces, and bisection is already very cheap.

Abstention is where intervention is unambiguously better. Pointed at runs that did not fail, counterfactual methods decline to name anyone 97.8-100% of the time. Every heuristic names a suspect 100% of the time, because a heuristic always has a "last span" to point at. A tool that always produces a culprit manufactures suspects.

4.6 The findings replicate on a real LLM agent

Everything above was measured on traces produced by rule-based Python. The same pipeline was then run with Claude Haiku 4.5 making the agent's decisions – same tools, same tasks, same faults, same loop guard and step budget, with only `_propose` replaced, so any difference is attributable to the policy and not the scaffolding. 44 failed traces (33 silent, 11 overt), 55 recovered, 5 clean. The agent solved 5/5 unfaulted tasks and emitted 4 unparseable replies across ~870 decisions.

method	scripted agent (n=547)	LLM agent (n=44)
cf_bisect	1.000 @ 3.74 replays	1.000 @ 3.73 replays
llm_trace_judge	0.543	0.500
first_error	0.177 (silent 0.000)	0.136 (silent 0.000)
output_anomaly	0.165	0.159
last_span	0.000	0.000

The method transfers. Counterfactual bisection holds at 1.000 with essentially identical cost – 3.73 replays against 3.74 – on traces generated by a model rather than by rules. `first_error` again scores exactly zero

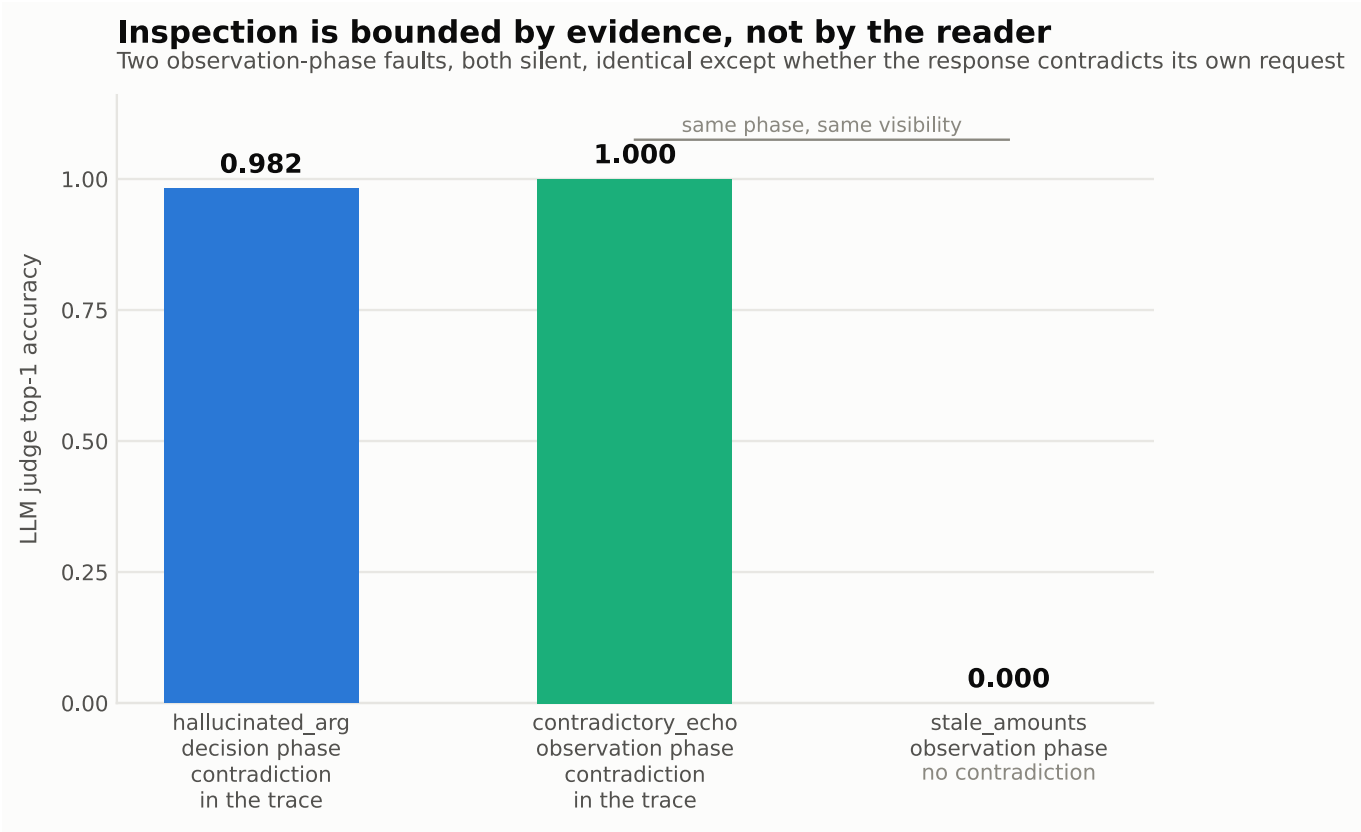
on silent faults. The ordering of every method is preserved.

Two differences are worth recording.

The LLM agent is more robust than the scripted one. It recovered from 55.6% of injected faults, against the scripted policy's 45.6%. A real model sometimes routes around corruption that a credulous rule-based agent swallows – which means the scripted arm, if anything, *overstates* how damaging silent faults are. That is the right direction for a limitation to point.

The judge's silent/overt parity may not hold, but this cannot be settled here. On LLM-agent traces the judge scored 0.424 on silent faults and 0.727 on overt – a gap where the scripted arm had none. With 11 overt traces the confidence intervals are [0.27, 0.59] and [0.43, 0.90]; they overlap heavily. The honest statement is that this arm is underpowered on that comparison and the question stays open, not that the parity reverses.

4.7 Why inspection plateaus: contradiction, not visibility



The 0.54 ceiling had no mechanism attached to it, and the obvious candidates were wrong. It is not fault visibility: the judge scores 0.545 on silent faults and 0.537 on overt, no gap. It is not trace length, which is non-monotonic once fault type is controlled for. Breaking accuracy down by fault exposed a clean split instead:

fault sits on	judge accuracy
decision side (the agent asked for the wrong thing)	0.921 [0.87, 0.95]
observation side (the tool returned the wrong thing)	0.360 [0.31, 0.41]

The proposed mechanism: a decision fault creates an *internal contradiction*. The agent requested `V1007` when an earlier span established `V1003` – two spans in the trace disagree, and reading is enough to find it. An observation fault produces data that is **externally wrong but internally consistent**: a tool returned 8,857 cents and nothing else in the trace says what it should have been. There is no contradiction, so there is nothing to read.

That pattern was found by looking at the data, which makes it a hypothesis and not a result. So it was turned into a prediction on a fault that did not exist yet:

An **observation-phase** fault engineered to contradict its own request should score like a decision fault (~ 0.9), not like its phase-mates (~ 0.36). If phase is what matters it will score ~ 0.36 ; if contradiction is what matters it will score ~ 0.9 .

`contradictory_echo` serves another vendor's invoices *and* echoes that vendor's id, so the response visibly disagrees with the request that produced it. The data is exactly as wrong as `stale_amounts`; the only difference is that the trace now contains the evidence.

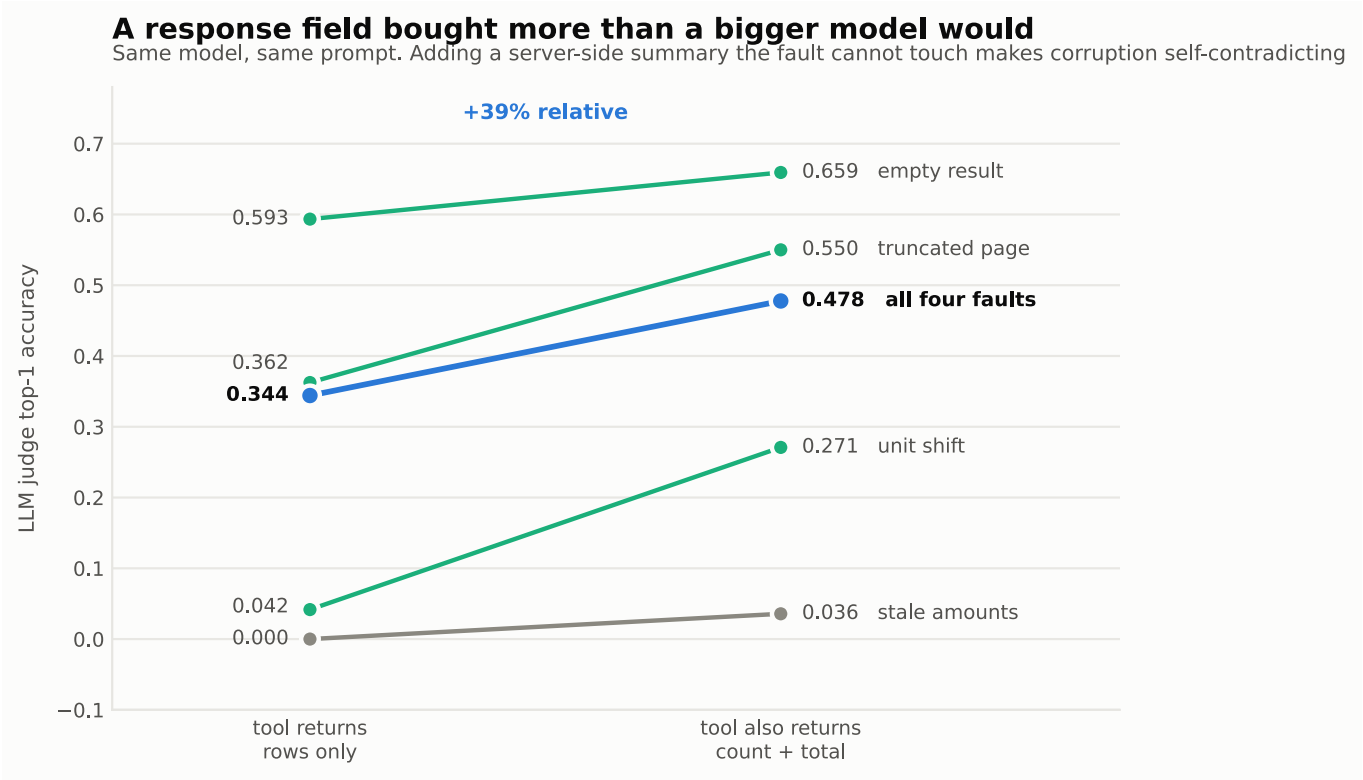
fault	phase	contradiction?	judge	replay
<code>hallucinated_arg</code>	decision	yes	0.982	1.000
<code>contradictory_echo</code>	observation	yes	1.000 [0.95, 1.00]	1.000
<code>stale_amounts</code>	observation	no	0.000 [0.00, 0.12]	1.000

Two observation-phase faults, both silent, identical in every structural respect except the presence of a contradiction: **1.000 against 0.000**, with non-overlapping intervals. Replay is unaffected at 1.000 throughout.

The ceiling on inspection is the presence of evidence in the trace, not the capability of the reader. A judge cannot detect a wrong number that nothing contradicts, and no better judge will, because the information required is not there. Intervention is not bounded this way: it does not search the trace for evidence, it generates evidence by re-running the agent.

One honest caveat: `contradictory_echo` was designed to be maximally contradictory, so 1.000 is an upper bound rather than a typical value. The result that carries the weight is the *contrast* with `stale_amounts` at 0.000, which holds phase and visibility fixed.

4.8 The diagnosis implies a treatment: instrumenting the tool



If inspection is bounded by evidence rather than by the reader, then adding evidence to the trace should raise the ceiling without touching the judge, the prompt or the model. That is a testable consequence, not a corollary, so it was tested.

`list_invoices` was instrumented to return a server-side `n_matching` and `total_matching_cents` beside the rows, computed from the true result set so a fault that rewrites the rows leaves them untouched. A silent corruption then disagrees with a summary inside its own response. Everything else was held fixed: same world seed, tasks, faults, injection points, judge and prompt.

fault	baseline	instrumented
all four	0.344 [0.29, 0.41]	0.478 [0.42, 0.54]
unit_shift	0.042	0.271
truncated_page	0.362	0.550
empty_result	0.593	0.659
stale_amounts	0.000	0.036
counterfactual replay (control)	1.000	1.000

A 39% relative improvement in a judge, bought by changing a tool's response shape rather than the model. Replay is unchanged, as predicted – it never needed the evidence because it generates its own.

`stale_amounts` is the interesting failure, and it refines the mechanism. Instrumentation did put the evidence in the trace, and the judge still could not use it. The difference is what checking the contradiction costs. `empty_result` and `truncated_page` reduce to comparing a count against the rows in hand; `unit_shift` is an order-of-magnitude mismatch visible at a glance. Catching `stale_amounts` means summing rows across several pages and comparing against a total when the drift is only 5-25%. The evidence is present; the arithmetic is not free.

So the sharper statement is that **a contradiction has to be cheap to check, not merely present**. That is a more useful claim than the one it replaces, because it says what kind of instrumentation pays: emit summaries that make corruption visible by counting or by magnitude, not ones that require exact arithmetic across spans to reconcile.

4.9 Using Phoenix as a user, not as a span store

The first version of this benchmark shipped with its own runner: sweep a corpus, score each method, print a table. It worked, and it was the wrong call. Phoenix is an experimentation product, and a ~150-line custom harness that runs a task over a corpus, scores it, stores results and compares methods is a reimplementaion of the feature. Writing it meant never finding out what that feature does well.

The benchmark now runs both Phoenix workflows.

Observability. The agent exports OpenInference spans over OTLP. Localization reads them back through `phoenix.client` and writes its verdict as a span annotation with `annotator_kind="CODE"`, so the accusation lands on the span it accuses and is filterable apart from human and LLM annotations. Round-trip fidelity is verified rather than assumed: localizing traces read back out of Phoenix agrees with localizing them in-process on 100% of cases. That check caught a real bug – an early read returned *partial* traces, because OTLP export and Phoenix ingestion are both asynchronous, and a partial trace silently corrupts localization since replay reconstructs the agent's steps from the spans it can see.

Experimentation. Failed traces become dataset examples: input is what a localizer may see, output is the known culprit span, metadata carries fault name, visibility and phase so results can be sliced in the UI. Each localizer is an experiment task; `correct_span`, `reciprocal_rank`, `blame_distance` and `replay_cost` are evaluators. A second dataset of non-failing runs scores abstention, where naming nobody is correct.

Two things this bought that the custom runner did not. Results became inspectable per example – you can open the trace where `first_error` failed and read what it accused instead – and comparison became the product's job, so adding a method later is one more experiment rather than a new column in a script.

The friction worth recording, because anyone doing this will hit it: a dataset example is JSON, but a localizer needs a live `Trace`, and the replay methods additionally need a world and an armed fault injector that cannot be serialized. The example therefore carries the *recipe* – case id, fault, target step, seed – and the task

rehydrates the case locally. That keeps the dataset honest, holding only what a Phoenix user could actually store, at the cost of the task needing the benchmark package importable.

5. The process, honestly

Seven times an analysis changed a claim rather than confirming it – five flaws in the harness, one in a reported statistic, and one contamination caught in a final audit. Each is in `EXPERIMENT_LOG.md` with the run that caught it.

Degenerate failures. The first fault sweep produced 161 failures per fault at a median of 49 spans, with no answer — the agent retried forever until the step cap. Trace length, not the fault, would have become the dominant signal any localizer keyed off. Fixed by giving the policy a loop guard and a step budget, both of which real agents have.

An incoherent signal. On the abstention experiment, counterfactual methods abstained 0% under the gold-based signal. That was a definition bug: on a run that already succeeded, "is the answer correct after repair?" is trivially true at the first probe, so the method confidently blamed span 1 for a failure that never happened. Fixed by requiring the answer to have *moved* as well as be correct.

A nondeterministic environment. Counterfactual replay scored 0.952 under the label-free signal. Rather than report the 5-point gap, I asked *where* it failed: all 26 misses came from one fault type, and every one blamed a span upstream. That fault drew its drift from a shared stateful RNG, so replay ran in a *different* broken environment than the original run. Fixed; label-free accuracy went to 1.000. **Top-1 accuracy alone said "95%, good enough" and hid the bug. The direction-of-error breakdown made it obvious in one table.**

A cache that could never hit. Span ids were random uuids regenerated on every corpus build, and they are rendered into every judge prompt as the handle the model answers with. Identical traces produced different prompts each run, so the response cache never hit across processes. This cost ~\$1.03 in unusable API calls before the `cached 0, new 100` line exposed it. I had claimed in the cache's own docstring that re-runs were free without ever verifying a cross-process hit. Fixed by deriving ids from a stable run id; the corpus is now byte-reproducible, and the judge experiment subsequently ran at **\$0.000 with 1,346 cache hits.**

The pattern is consistent: every one of these was found by asking a *second* question about a result rather than accepting the headline number. Three of the four changed a figure that would otherwise have been published.

6. What this does not show

- **The LLM-agent arm is small.** 44 failed traces against the scripted arm's 547, so its confidence intervals are wide and it settles replication, not fine-grained comparisons. It was run on one model (Claude Haiku 4.5).

- **Exactly one fault per run.** Real failures sometimes have several interacting causes. This also means the label-free and gold-based signals coincide *by construction* — any change that helps also corrects — so "label-free is free" is a claim about this benchmark, not in general.
- **The repair oracle is a probability, not a model.** Its real-world reliability is unmeasured, which is why §4.3 is a sweep rather than a number.
- **Replay requires a re-executable environment.** A trace records what did happen; a counterfactual asks what would have. Read-only tools qualify; a tool that charges a card does not. This is an explicit protocol in the codebase rather than a buried assumption.
- **One judge model, one size.** Whether a frontier model closes the remaining gap to 1.000 is untested.

7. Where this leaves the product

For a Phoenix user with re-executable tools, the recommendation is specific: **prefix bisection with a sampled repair**, on the label-free signal, writing verdicts back as span annotations. It is ~ 3.7 replays per trace, depth independent, and it abstains on healthy runs.

The judge result reframes the pitch, and the honest framing is narrower than the one I started with. It is **not** that judges are blind – a one-call judge reaches 0.543, six times chance, with no silent/overt weakness, and needs no replay environment. For triage that may well be enough.

The case for intervention rests on three measured things, none of which is a claim about judge quality:

1. **It resolves the last 46 points.** 1.000 against 0.543, on both fault classes and on both agent implementations.
2. **Its errors are correctable and inspection's are not.** A judge that misses is typically three spans away in both directions, so there is no bias to exploit and no cheap post-hoc fix. A replay either moves the outcome or it does not.
3. **It can abstain.** Inspection has no way to express "nothing here", because there is always a most-suspicious span. On runs that did not fail, intervention declines 97.8-100% of the time; every heuristic names a suspect every time. For a debugging tool pointed at whatever a user flags, that is the difference between an assistant and a generator of suspects.

The cost of intervention is not the replay count – 3.74 is cheap. It is the requirement for a re-executable environment, which is a real constraint that rules out side-effecting tools entirely.

The layered product is probably right: judge for triage, replay for confirmation. The judge is cheap enough to run on everything and good enough to rank; replay is precise enough to be believed and honest enough to stay quiet.

The instrumentation result (§4.8) is the most directly actionable thing here: a team that cannot adopt replay, because its tools are not safely re-executable, can still buy a 39% relative improvement in a judge by having those tools return redundant summaries. That costs a response field, not a model upgrade.

The most valuable next experiment is a larger LLM-agent arm across several models. The 44-trace replication shows the method transfers; what it cannot show is whether a more capable agent – one that already recovers from 55.6% of faults – changes which faults matter in practice. The second is a mechanism for the 0.54 plateau, which this work measures but does not explain.